WAVE(Wide Area Verified Exchange)

A Decentralized Authorization System for IoT via Blockchain Smart Contracts

Michael P Andersen, John Kolb, Kaifei Chen, Gabe Fierro, David E. Culler, Raluca Ada Popa

Introduction

- Existing systems rely on **central authority**.
  - If an attacker compromises this party, they can subvert the authorization policy of the entire system.
- In WAVE (Wide Area Verified Exchange), this party is replaced with a blockchain.
- Using blockchains for various tasks in IoT has already been proposed. Such systems place **user or device data** onto the blockchain
  - Inefficient and expensive
- WAVE uses the blockchain only for storing **permissions** (essentially “metadata”) and **not data**, and does not access the blockchain upon critical data path operations.

Constraints

- Heterogeneity and interaction across trust domains.
  - Alice can have certain permissions in the context of her office and other permissions in the context of her home. She might be able to control the lights in a room, but not to unlock that room.
- Non-interactivity
  - IoT devices are intermittently connected. Hence, when granting access to an entity, we should not expect that the entity is online.
  - When the entity comes online, it should be able to exercise its access without further interaction.

Constraints

- Out-of-order delegation
  - Bob is giving an assistant access to everything work-related.
  - Later, the building admin gives access to Bob to a new thermostat.
  - The assistant should get access immediately to the thermostat. Bob essentially gave access to the thermostat before he had access to it.
  - When combined with non-interactivity, the assistant must gain access to the thermostat *without* Bob coming online to help.
- Constrained resources
  - Many IoT devices have constrained computing, storage and energy resources

Goals

- Wave must provide an integrated way to express authorization relations across trust domains and applications.

- Wave must not rely on a central point of trust. The IoT network can cross administrative domains with devices and services persisting beyond periods of ownership or vendor lifespan. No party can be trusted by the entire world with authentication, and any central party represents a point of attack.

- The system should provide protection against DDoS attacks.

Goals

- The system should minimize privacy leakage. With a global network, the attacker has the ability to observe actions in the network and infer private information.

- While removing all sources of leakage in IoT is a vastly unsolved question, one should attempt to protect the more sensitive pieces of information, such as message streams and resource hierarchies.

  - For example if an administrator delegates to Alice access to the resource URI “BuildingNorthGate/Floor9/Room25”, an attacker essentially learns where Alice works.

WAVE

Authorization layer	Entities, DoTs, Namespaces
Syndication layer	Resources, Publish / Subscribe
Overlay layer	Routers (brokers), agents IoT devices, blockchain, servers

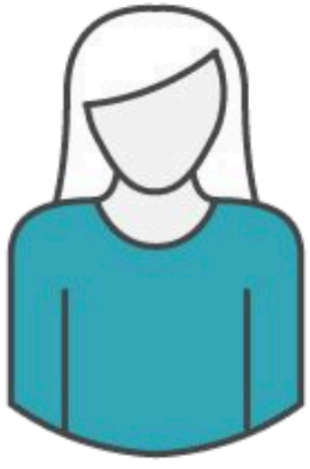
WAVE

Authorization layer: Entity

$<E_{sk}, E_{vk}>$

A keypair for signing and verifying

- Identified by E_{VK} e.g l0hKkvaVyRDqf_lwt93WJC_a9Zu2F3l61Au6fZtIsCU=
- Optionally identified by a globally unique, immutable alias e.g mike19
- Represents the holder of the signing key:
  - IoT device
  - Participant
  - Services



WAVE

Authorization layer: Namespaces & Resources

A namespace is controlled by an entity E_{ns} and is a collection of resources:

namespace/resource_path

- All resource URIs begin with the E_{ns} of the namespace entity (or its alias)
- E_{ns} has full permissions on all resources within the namespace

alicehome/hvac/thermostat/setpoint
alicehome/security/door/islocked
caiso/pricing/zone25/electricity

WAVE

Authorization layer: Delegation of Trust

$<E_{from}, E_{to}, URI_{resource}, Permissions, Sig_{Efrom}>$

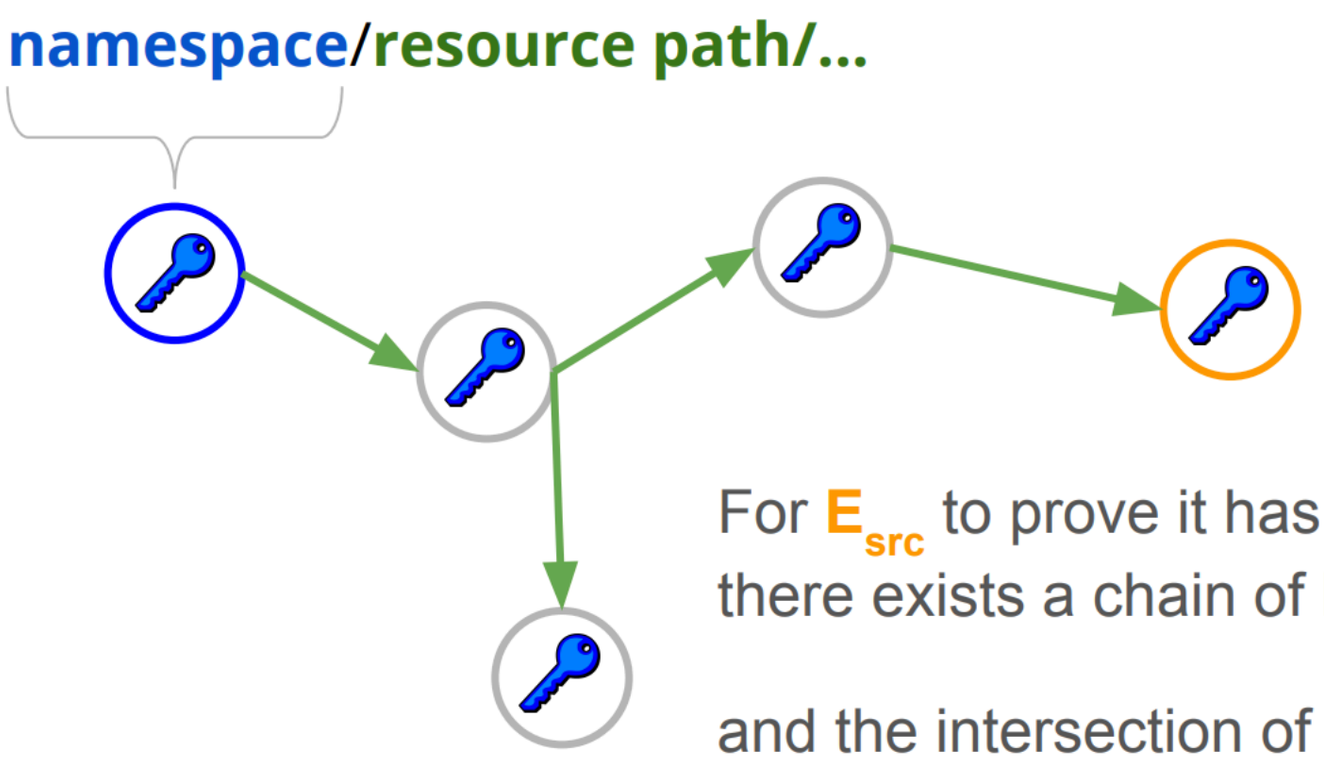
For other entities to obtain permissions on a resource, they must receive them via a *delegation of trust (DoT)*

A DoT is *useful* if the granter E_{from} itself has the permissions.

This object is public and discoverable

WAVE

Authorization layer: DoT graph



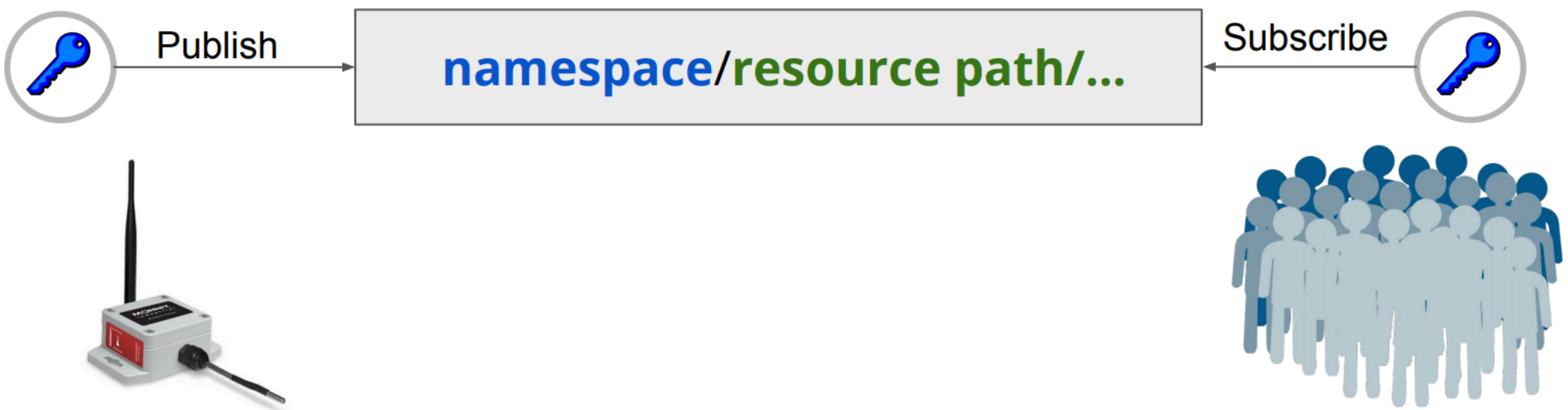
For E_{src} to prove it has **P** on a **URI**, it is sufficient to show there exists a chain of DoTs, end to end, from E_{NS} to E_{SRC}

and the intersection of the permissions granted by DoTs on this chain is greater than or equal to **P**

WAVE

Syndication tier

- An entity can **subscribe** to a resource
- It will receive all messages **published** to that resource
- Same pattern as other pub/sub used in the IoT space



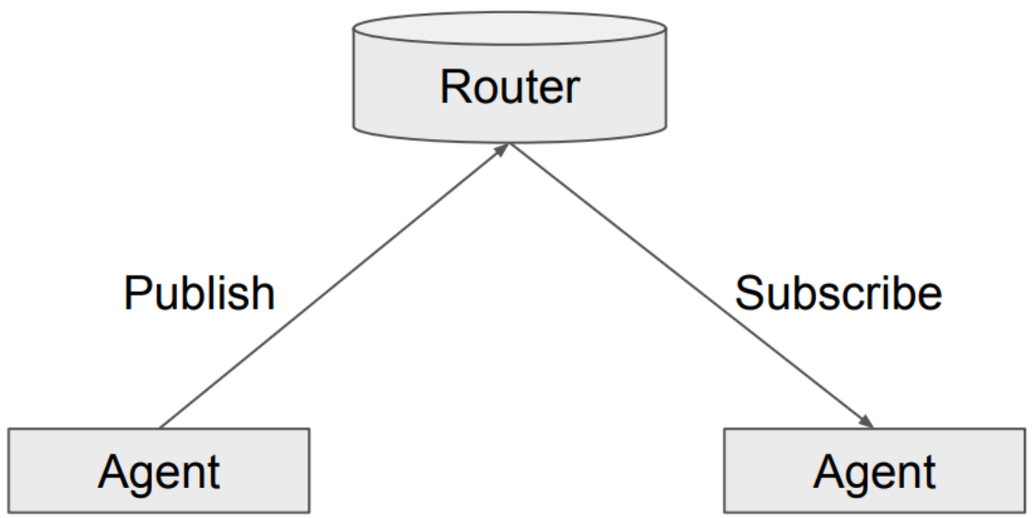
WAVE

Syndication tier: messages

- A message consists of:
  - A Resource URI
  - The payload to publish to the URI
  - A chain of DoTs proving the message sender is authorized
  - A signature by the sender verifying the message has not been tampered or forged (authentication)

WAVE

Overlay tier: Routers and Agents



Agent: acts on behalf of an Entity

- Builds proofs for entity
- Encodes and signs messages
- Validates incoming messages
- Participates in block chain

Router: routes messages for a namespace

- Verifies proofs
- Verifies messages
- Forwards published messages to subscribers
- Participates in block chain

Overlay Tier

Four contracts

WAVE object inspection:

Validate signatures
Decode packed objects

Precompiled for speed

Registry:

Store E_{vk} + metadata
Store DoT
Index objects for access

Affinity:

Store E_{NS} -> E_{DR} map
Store E_{DR} -> IP address

Like DNS but no central authority

Aliases:

Store E_{vk} <-> E_{alias} mappings

Like DNS but immutable

Proof and Verification in Overlay Layer

Agents and routers work with a representation of permissions called a *DoT chain*: a proof that an entity E_{vk}^{prover} can interact with a resource URI or URI pattern on a namespace E_{vk}^{ns}. It manifests as a path through the DoT graph leading from E_{vk}^{ns} to E_{vk}^{prover}. The permissions granted by the chain are the intersection of those granted by its DoTs ; expiry or revocation of any entity or DoT in the chain invalidates the whole.

A DoT chain is serialized as a list of DoT hashes. To verify the chain a router/agent first *elaborates* the chain by resolving each hash into the full DoT object and verifying it. It then verifies that every DoT's "from" is the "to" of the previous, the intersection of the permissions is a superset of those claimed by the proof, and the TTL imposed by the DoTs in the chain is not exceeded by the length of the chain.

A *message* encapsulates an action on a resource by combining a URI, the action, a DoT hash chain terminating at E_{vk}^{sender} proving the action is permitted on the URI, and a signature by E_{sk}^{sender}. A message is verified by checking the signature and then verifying the proof.

PDoT: Protected DoT

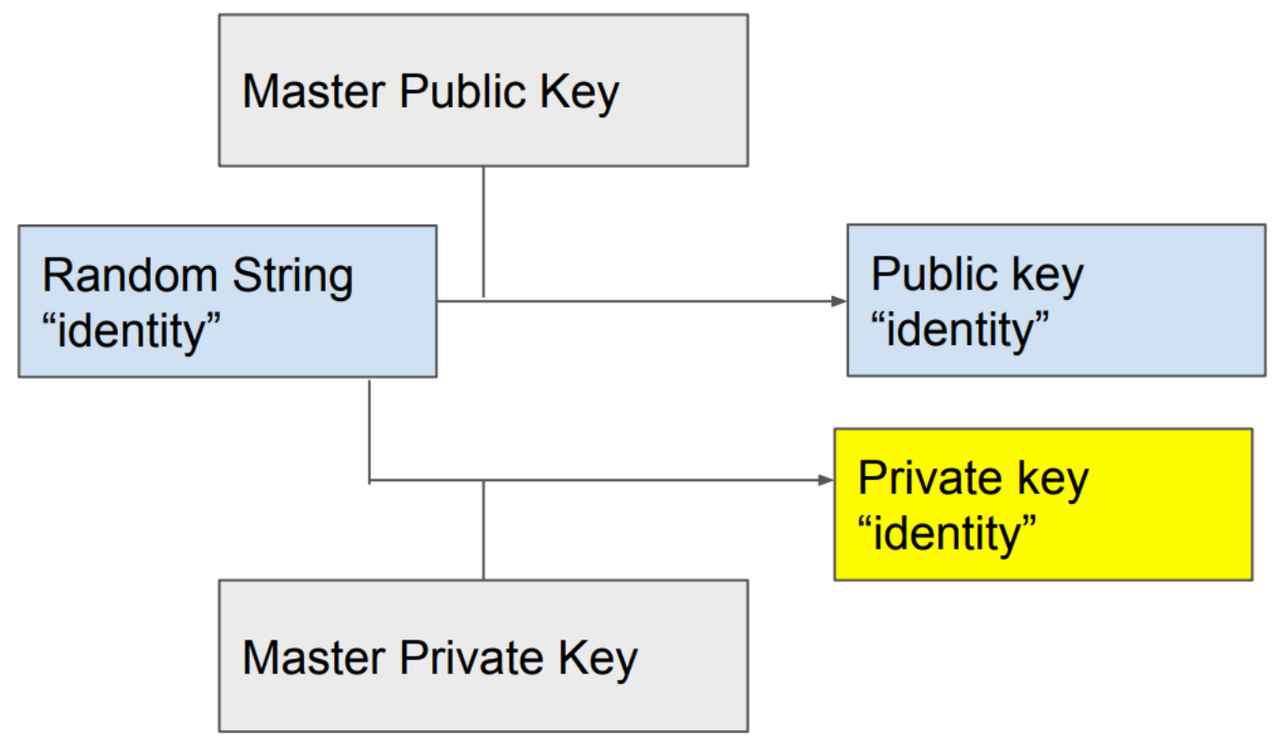
- Privacy issues
  - Permissions grant access to resources
  - Resource URIs contain potentially sensitive information

<alice, bob, /gndfloor/lock/**samsunglock**/openstate, sub>

<ed, phil, /**17mlk**/r&d/**project_infinity**/grav_feeds, pub>

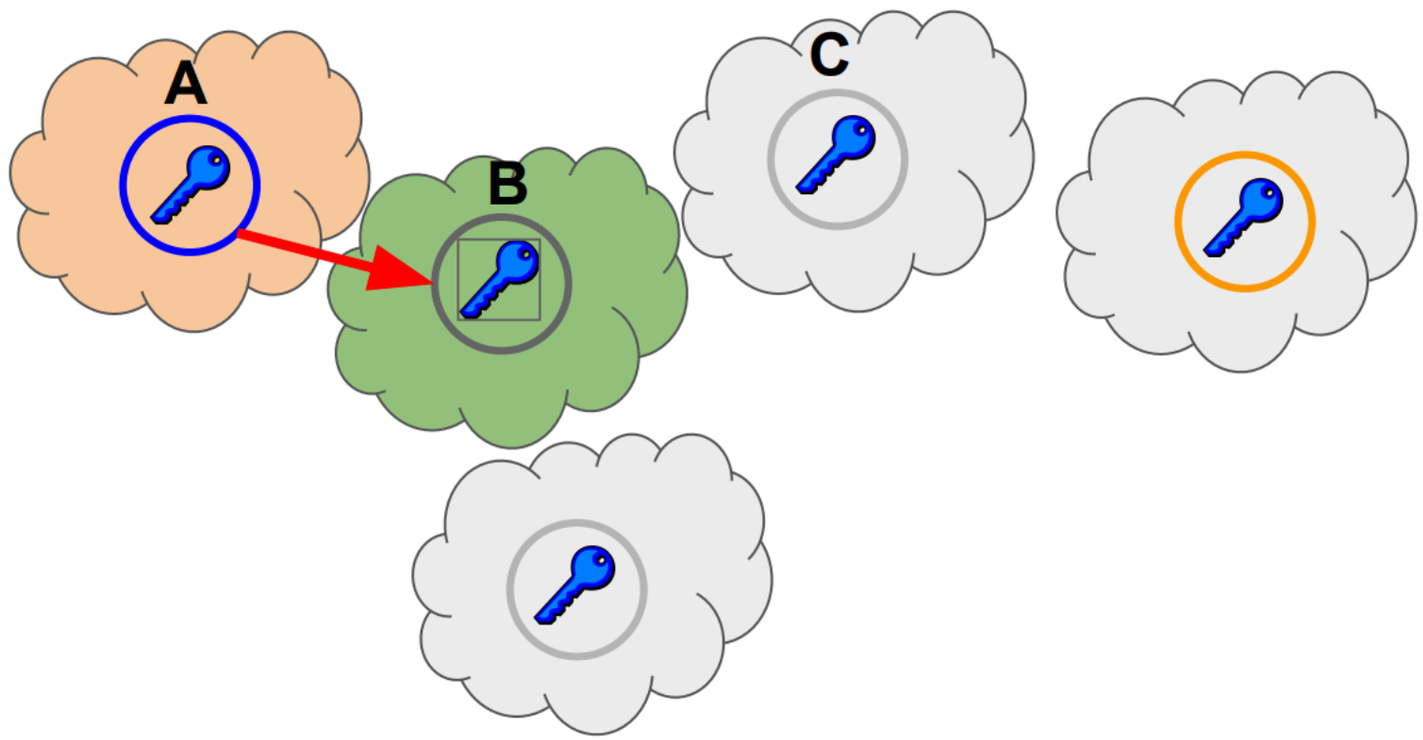
PDoT: Protected DoT

- ID-based Encryption (IBE) private key:
  - Is only used for encrypting DOTs
  - Only in a single namespace
  - Only granting a specific set of permissions
    - That you were granted access to
- So only a minimal set of information is revealed.



PDoT: Protected DoT

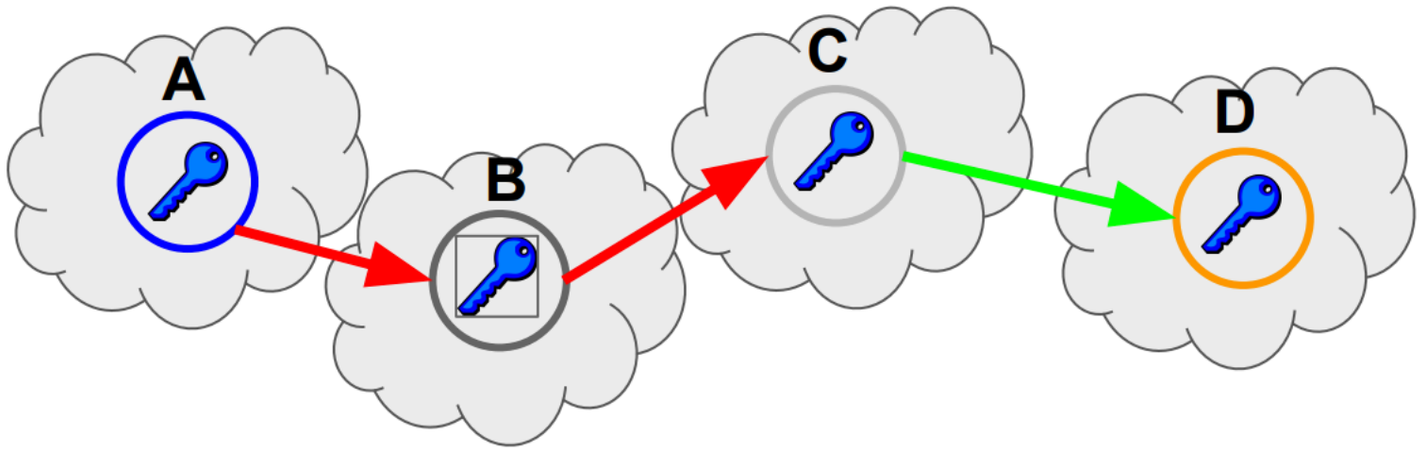
Protected DoTs



Encrypt **this DoT** in the **recipient IBE system** using the **namespace and permissions** as the “identity” but also include the PRIVATE KEY in the **source IBE system** generated with the SAME identity

PDoT: Protected DoT

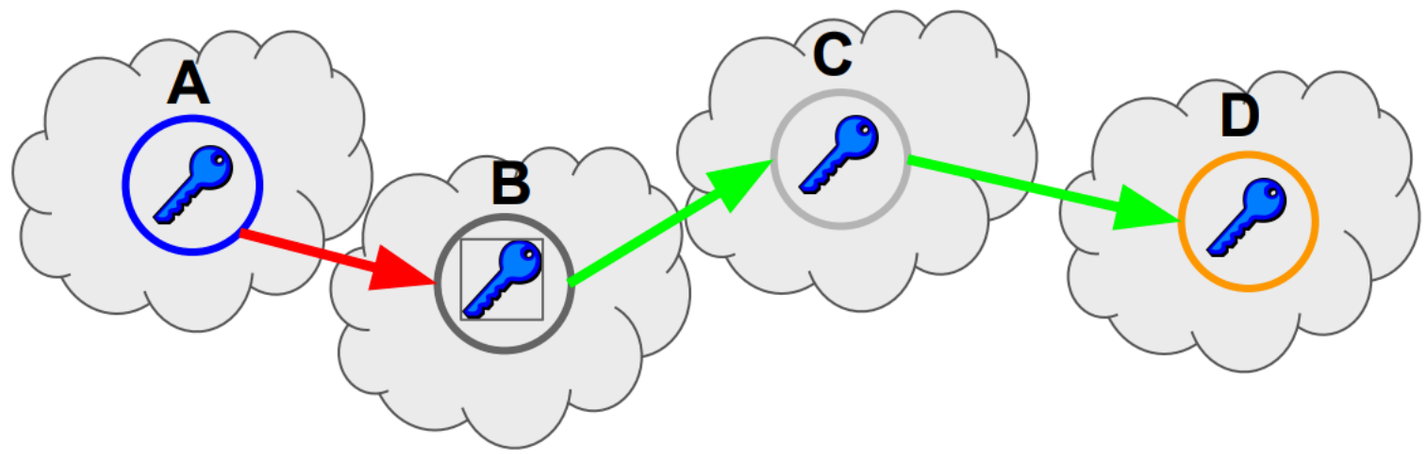
Protected DoTs



D can trivially decode the DoT from C to D because its under **D**'s key

PDoT: Protected DoT

Protected DoTs



D can trivially decode the DoT from C to D because its under **D**'s key
Then **D** learns **C**'s private key, so can decode BC

DDoS Protection

- Routers may employ DDoS mitigation strategies:
  - blacklisting entities that attempt too many unauthorized actions.
  - As registering entities requires blockchain interaction, the rate at which the attacker can create new entities is severely limited, rendering an attack at this layer ineffective.
- An attacker might attempt lower-level attacks, such as a layer 3 bandwidth DDoS.
  - To mitigate this, we inspect the TLS handshake.
  - Traffic categorization between valid and malicious is used to configure upstream routers.
  - Any session can be easily categorized as malevolent with no false positives by inspecting the first 96 bytes of the stream (containing the E^{agent} and a signature of the TLS certificate)and comparing against the permission graph.

WAVE

- Tests
  - CPU, memory and bandwidth usage on a variety of platforms under different scenarios (idle, normal, attack) and different internet characteristics (latency, speed)
  - City-scale (millions of people) emulation drawn from public San Francisco data
  - Robust DDOS protection due to rapid, accurate traffic identification
  - >400 days of deployment across a handful of namespaces, hundreds of devices and tens of thousands of resources